

ЗВІТ

з лабораторної роботи № 5

Обробка помилок, винятків і логування

Реквізит	Значення
Дисципліна	Конструювання програмного забезпечення
Проект	LibraryDesk — облік видачі книг
Варіант	№ 2
Виконав	Гордєєв Дмитро
Група	ІПЗ
Дата	8 жовтня 2026 року

Гілка репозиторію: lab05-error-handling

1. Мета роботи

Мета роботи — розділити очікувані помилки введення, порушення правил предметної області та збої зовнішнього середовища. У застосунку побудовано барикаду між текстовим інтерфейсом і типізованим ядром, додано `Result<T>` та `DomainRuleException`. Також виправлено антипатерни обробки винятків, підключено структуроване логування і гарантоване звільнення файлового ресурсу.

2. Вихідні дані

LibraryDesk — .NET 8 застосунок для обліку видачі книг. Документом є `Loan`, позицією документа — `LoanItem`, учасником — `Reader`, а об'єктом обліку — `Book`. Продовжено код лабораторних робіт № 1–4 без створення нового рішення.

Тип	Призначення	Ключові правила
Loan	Формуляр видачі	Стани Draft, Active, Overdue, Returned, Cancelled, Lost
LoanItem	Позиція формуляра	ISBN не порожній; Days > 0; DailyRate ≥ 0
Reader	Читач бібліотеки	Id > 0; ім'я і пошта заповнені
LoanService	Прикладні сценарії	DI, предметні винятки та журналювання

Барикада проходить між `LibraryDesk.App` та `LibraryDesk.Core`. Зовні ядра перебувають рядки, культура введення і повторні запити; усередині — лише перевірені `LoanItem` та інші типізовані об'єкти.

3. Завдання 1. Класифікація помилок

№	Ситуація	Клас	Місце	Стратегія	Повідомлення
1	«два» замість кількості	Користувача	LoanItemParser	Result.Fail; повтор	Кількість — ціле число > 0
2	Порожній ISBN	Користувача	Парсер / каталог	Result.Fail; без змін	Вкажіть ISBN книги
3	Некоректний тариф	Користувача	LoanItemParser	Result.Fail; пояснення	Тариф — невід'ємне число
4	null-залежність сервісу	Програміста	LoanService.ctor	ArgumentNullException	Внутрішня помилка INIT-01
5	Return для Draft	Програміста	Loan.Return	DomainRuleException	Внутрішня помилка STATE-01
6	Від'ємна вартість	Програміста	LoanService.Total Of	Error; припинити	Внутрішня помилка CALC-01
7	Файл аудиту зайнятий	Середовища	FileAuditLog	IOException на межі	Журнал зайнятий; повторіть
8	Немає прав на запис	Середовища	FileAuditLog	Повідомити; код	Немає прав на журнал
9	Збій сховища	Середовища	Repository / Register	Error; throw зі стеком	Формуляр не збережено

Таблиця 1. Повна класифікація міститься у docs/errors.md разом із повідомленнями користувачеві. Стратегія навмисно різна: користувацький ввід є очікуваною гілкою, помилка програміста має виявлятися швидко, а збій середовища потребує контекстного журналу на межі застосунку.

4. Завдання 2. Барикада і валідація

До рефакторингу ядро могло б саме розбирати текстові значення:

```
// БУЛО: сирі рядки потрапляють у ядро
public void AddItem(int loanId, string isbn,
                   string days, string rate)
{
    int parsedDays = int.Parse(days);
    decimal parsedRate = decimal.Parse(rate);
    AddItem(loanId,
            new LoanItem(isbn, parsedDays, parsedRate));
}
```

Лістинг 1 — стан без барикади

```
// СТАЛО: розбір виконується у LibraryDesk.App
public static Result<LoanItem> Parse(
    string isbn, string days, string dailyRate)
{
    if (!int.TryParse(days, NumberStyles.Integer,
        CultureInfo.CurrentCulture, out int parsedDays)
        || parsedDays <= 0)
        return Result.Fail<LoanItem>(
            "Кількість днів має бути цілим числом більше нуля.");

    if (!decimal.TryParse(dailyRate, NumberStyles.Number,
        CultureInfo.CurrentCulture, out decimal parsedRate)
        || parsedRate < 0m)
        return Result.Fail<LoanItem>("Некоректний тариф.");

    return Result.Ok(
        new LoanItem(isbn.Trim(), parsedDays, parsedRate));
}
```

Лістинг 2 — скорочений LoanItemParser після побудови барикади

Метод `Loan.AddItem` приймає готовий `LoanItem` і не знає про формат консолі. Десятковий роздільник визначається `CultureInfo.CurrentCulture`. Перевірка продукційних каталогів ядра не знаходить `int.Parse` або `decimal.Parse`.

5. Завдання 3–4. DomainRuleException і Result

```
public class DomainRuleException : Exception
{
    public DomainRuleException(string rule, string message)
        : base(message) => Rule = rule;

    public DomainRuleException(string rule, string message,
                                Exception innerException)
        : base(message, innerException) => Rule = rule;

    public string Rule { get; }
}
```

Лістинг 3 — предметний виняток з ідентифікатором правила

```
public readonly struct Result<T>
{
    internal Result(bool ok, T? value, string error)
    {
        IsSuccess = ok; Value = value; Error = error;
    }
    public bool IsSuccess { get; }
    public T? Value { get; }
    public string Error { get; }
}

public static class Result
{
    public static Result<T> Ok<T>(T value) =>
        new(true, value, string.Empty);
    public static Result<T> Fail<T>(string error) =>
        new(false, default, error);
}
```

Лістинг 4 — тип результату для очікуваних помилок

```
public Loan GetRequiredLoan(int loanId)
{
    Loan? loan = _repository.GetById(loanId);
    if (loan is null)
        throw new DomainRuleException(
            "loan.exists", $"Формуляр {loanId} не знайдено.");
    return loan;
}
```

Лістинг 5 — null перетворюється на виняток у бізнес-сценарії

Метод	Рішення	Причина
ILoanRepository.GetById	Залишити Loan?	Відсутність запису — нормальний результат пошуку
InMemoryLoanRepository.GetById	Залишити Loan?	Реалізація відповідає контракту репозиторію
LoanService.GetRequiredLoan	DomainRuleException	Операція вимагає наявного документа

`Result<T>` використано для очікуваних помилок введення. Він містить `IsSuccess`, `Value` та `Error`, тому консольний шар показує пояснення й повторює спробу без аварійного завершення.

6. Завдання 5. Виправлення антипатернів

Антипатерн	Місце до	Заміна	Усунутий наслідок
Порожній catch	Before.Load	Не перехоплювати	Збій не зникає без сліду
catch (Exception)	Before.Cancel	catch (IOException)	Помилки програміста не маскуються
throw exception	Before.Cancel	innerException / throw	Початковий стек збережено
Parse у try/catch	Before.CountValidDays	int.TryParse	Ввід не створює виняток

Таблиця 2 — відповідність стану «до» та фінального виправлення

```
// ДО: широкий catch і втрата стека
catch (Exception exception)
{
    Console.WriteLine("Щось пішло не так");
    throw exception;
}

// ПІСЛЯ: конкретний збій і збережена причина
catch (IOException exception)
{
    throw new DomainRuleException(
        "loan.storage",
        $"Не вдалося скасувати формуляр {loanId}.",
        exception);
}
```

Лістинг 6 — виправлення широкого перехоплення і втрати стека

```
// ДО: виняток керує звичайною гілкою
try { int days = int.Parse(row); }
catch (FormatException) { }

// ПІСЛЯ: очікувана перевірка без винятку
if (int.TryParse(row, out int days) && days > 0)
    valid++;
```

Лістинг 7 — заміна винятку як керування потоком

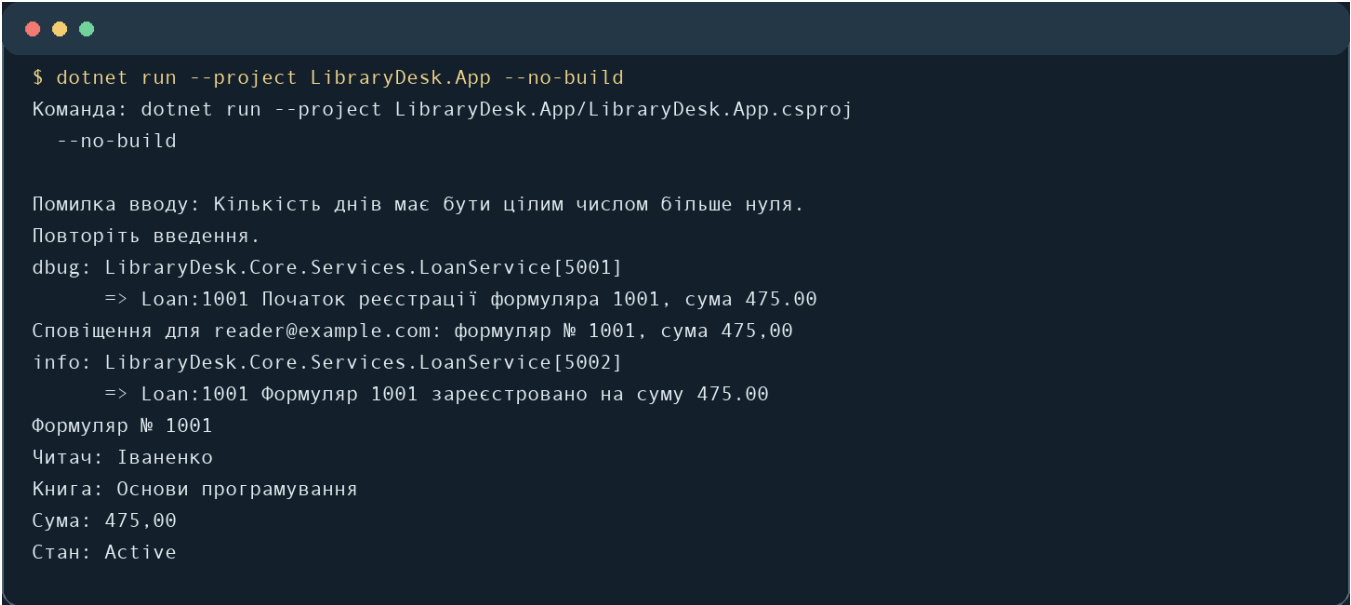
Погана реалізація збережена комітом a503ce1 «antipatterns: before», після чого файл замінено виправленим LoanErrorHandling. Єдиний catch (Exception) залишився у Program.cs як остання межа процесу.

7. Завдання 6. Структуроване логування

LoanService отримує ILogger<LoanService> через конструктор. Повідомлення генеруються LoggerMessage і мають іменовані поля LoanId, Total, Rule та Status. BeginLoanScope додає контекст Loan:{LoanId} до всієї операції. У коді є рівні Debug, Information, Warning та Error; Critical використано у верхньому обробнику непередбачених збоїв.

```
using IDisposable? scope =  
    LoanServiceLog.BeginLoanScope(_logger, loan.Id);  
LoanServiceLog.RegisterStarting(  
    _logger, loan.Id, total);           // Debug  
LoanServiceLog.ZeroTotal(  
    _logger, loan.Id);                 // Warning  
LoanServiceLog.StorageFailure(  
    _logger, exception, loan.Id);      // Error  
LoanServiceLog.Registered(  
    _logger, loan.Id, total);          // Information
```

Лістинг 8 — рівні та контекст журналу



```
$ dotnet run --project LibraryDesk.App --no-build  
Команда: dotnet run --project LibraryDesk.App/LibraryDesk.App.csproj  
--no-build  
  
Помилка вводу: Кількість днів має бути цілим числом більше нуля.  
Повторіть введення.  
dbug: LibraryDesk.Core.Services.LoanService[5001]  
=> Loan:1001 Початок реєстрації формуляра 1001, сума 475.00  
Сповіщення для reader@example.com: формуляр № 1001, сума 475,00  
info: LibraryDesk.Core.Services.LoanService[5002]  
=> Loan:1001 Формуляр 1001 зареєстровано на суму 475.00  
Формуляр № 1001  
Читач: Іваненко  
Книга: Основи програмування  
Сума: 475,00  
Стан: Active
```

Знімок екрана 1 — помилка введення, повтор і структурований журнал

8. Завдання 7. using та IDisposable

```
public sealed class FileAuditLog : IDisposable
{
    private readonly StreamWriter _writer;
    private bool _disposed;

    public void Write(string operation, int loanId)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        _writer.WriteLine(
            $"{DateTimeOffset.Now:O};{operation};{loanId}");
        _writer.Flush();
    }

    public void Dispose()
    {
        if (_disposed) return;
        _writer.Flush();
        _writer.Dispose();
        _disposed = true;
    }
}
```

Лістинг 9 — скорочений FileAuditLog

У Program.cs журнал створюється через using. Навіть якщо наступна операція кине виняток, компілятор перетворить using на try/finally і викличе Dispose. Тест нижче навмисно перериває блок, а потім відкриває файл ексклюзивно.

```
Assert.Throws<InvalidOperationException>(
    () => WriteAndFail(path));

using FileStream reopened = new(
    path, FileMode.Open, FileAccess.ReadWrite,
    FileShare.None);
Assert.True(reopened.Length > 0);
```

Лістинг 10 — перевірка звільнення файла після винятку

Властивість	Результат
Власник ресурсу	FileAuditLog володіє StreamWriter
Гарантія	Dispose викликається при нормальному виході та винятку
Повторне Dispose	Безпечно завдяки прапорцю _disposed
Перевірка	Файл повторно відкривається з FileShare.None

9. Перевірка та історія комітів

```
$ dotnet build LibraryDesk.sln
Команда: dotnet build LibraryDesk.sln

LibraryDesk.Core -> LibraryDesk.Core/bin/Debug/net8.0/LibraryDesk.Core.dll
LibraryDesk.App -> LibraryDesk.App/bin/Debug/net8.0/LibraryDesk.App.dll
LibraryDesk.Tests -> LibraryDesk.Tests/bin/Debug/net8.0/LibraryDesk.Tests.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)
```

Знімок екрана 2 — збірка без попереджень і помилок

```
$ dotnet test LibraryDesk.sln --no-build
Команда: dotnet test LibraryDesk.sln --no-build

Пройдено: 14
Не пройдено: 0
Пропущено: 0
Усього: 14
```

Знімок екрана 3 — усі 14 тестів пройдено

```
$ git log --oneline main..lab05-error-handling
10927e2 feat(lab5): release audit resources deterministically
4844e61 feat(lab5): add structured logging and top-level handlers
a89a3e8 refactor(lab5): replace exception antipatterns
a503ce1 antipatterns: before
84909d6 feat(lab5): handle expected input errors with Result
14be248 refactor(lab5): enforce domain rules with exceptions
2329ce5 feat(lab5): add input barricade and error types
d54e221 docs(lab5): classify application errors
```

Знімок екрана 4 — окремі тематичні коміти завдань 1–7

10. Висновки

Найчисленнішими в LibraryDesk виявилися помилки користувацького введення та порушення станів формуляра. Найскладніше було помітити втрату початкового стека через throw exception, оскільки код зовні виглядав як повторне передавання тієї самої помилки. Барикада зробила межу застосунку явною: рядки перевіряються у LibraryDesk.App, а ядро отримує готові значення. Result<T> спростив нормальну реакцію на помилкове введення, тоді як DomainRuleException надав порушенням правил стабільні ідентифікатори. Структуроване логування додало придатні для пошуку поля та контекст однієї операції. using і IDisposable гарантують звільнення файла незалежно від способу завершення блоку. Після змін код став передбачуванішим і простішим для діагностики.

Підсумок виконання

Критерій	Стан
9 ситуацій, по 3 на кожен клас	Виконано
Барикада та типізоване ядро	Виконано
DomainRuleException і Result	Виконано
4 антипатерни та стан «до»	Виконано
4 рівні логування та BeginScope	Виконано
using, IDisposable і тест після винятку	Виконано
Build: 0 warnings / 0 errors	Виконано
Tests: 14 / 14	Виконано

Репозиторій

<https://github.com/kuum-oss/LibraryDesk>

Гілка: lab05-error-handling